PORTFOLIO DSBINFOPEPP01 PROJEKT: PROF. DR. DIRK SIMON (WiSe 2025/26): AUFGABENBESCHREIBUNG

VORBEMERKUNGEN

Das Portfolio dient zur praktischen Verwendung von Wissen zur Lösung realer Probleme aus den Übungen der Vorlesung **Einführung in die Programmierung mit Python**, die im Kurs erarbeitet wurden. Laden Sie das gesamte Portfolio bis zur Abgabefrist als finale Version des Portfolios, bestehend aus Abstract als PDF, nebst Software-Lösung „Jupyter-Projekt (Programmierressourcen mit Klassenhierarchie)“ der Phase 1 und der Phase 2.

Es gelten folgende Fristen und Abgabeorte:

1. Konzeptionsphase: Abgabe in Teams **(31.01.2026)**
2. Erarbeitungsphase / Reflexionsphase: Abgabe in Teams **(28.02.2026)**
3. Finalisierungsphase: Abgabe in Atlas / PebblePad **(31.03.2026)**

Hinweise:

Als Tools zur Bearbeitung kann folgendes zum Einsatz kommen: **Jupyter Notebook**, **PyCharm-Professional**, oder **Visual Studio Code**.

- Zur Erstellung der UML-Diagramme bieten sich die folgenden Tools an.
- LucidChart
- MIRO
- Visual Paradigm
- Draw.io

Möglich ist aber auch der Enterprise Architect von Sparx: Enterprise Architect (Sparx)

FORMALE KRITERIEN

Nichteinhaltung der folgenden Kriterien kann zu Punktabzug führen:

Das Portfolio, bestehend aus drei Phasen, soll die einzelnen Arbeits- oder Entwicklungsschritte in **nachvollziehbarer Weise** veranschaulichen.
Stellen Sie sicher, dass die **Abbildungen gut lesbar** sind, insbesondere vollständig beschriftet und hinreichend groß.

1. Phase (Konzeptionsphase) soll aus folgenden Inhalten bestehen:
    i. Das Ziel der Arbeit definieren.
    ii. Die Idee bestimmen und diese auf Tragfähigkeit überprüfen.
    iii. Das Konzept erarbeiten.
    iv. Einen Vorgehensplan erstellen.
    v. Die Methodik und das verwendete Tool diskutieren.

    Abgabe:

    1. Kurze Erläuterung des Konzepts in Form eines Textes von maximal einer halben DIN-A4-Seite
    2. Requirements-Engineering mit entsprechenden Diagrammen. Hierbei sollten bekannte Diagrammtypen, wie etwa Use-Case-Diagramm (Kontext-, Anwendungsfall-, Geschäftsprozessdiagramm) zur Darstellung der Fragen (Warum ist das Problem wichtig? Wen betrifft das Projekt? Was ist der Scope? Was ist System? Was ist Systemkontext? Was ist Systemgrenze?) verwendet werden.

       Zusätzlich kann hier auch ein Bereitstellungs- oder Verteilungsdiagramm verwendet werden.

2. Phase (Erarbeitungsphase / Reflexionsphase) soll aus folgenden Inhalten bestehen:
    i. Realisierung des Projekts / der Aufgabe.
    ii. Orientierung an den Zielen / an dem Projektplan
    iii. Beachtung der zur Verfügung stehenden Ressourcen
    iv. Erkennung von potenziellen Problemen, Risiken und Verbesserungsmöglichkeiten
    v. Überwachung des Status / Fortschritt

    Abgabe:

    1. Kurze Erläuterung zur Umsetzung in Form eines Textes von maximal einer halben DIN-A4-Seite (Erste Form der Umsetzung (noch nicht final))
    2. Requirements-Engineering mit entsprechenden Diagrammen. Hierbei sollten bekannte Diagrammtypen, wie etwa Aktivitäts-, Sequenz- und Klassendiagramm verwendet werden.

       Zusätzlich können hier auch noch weitere detailliertere statische oder dynamische Modellierungsmöglichkeiten verwendet werden.

3. Exemplarischer stark vereinfachter Code zur Illustration der Architektur. Die vollständige Implementierung ist erst in Phase III erforderlich.

3. Phase (Finalisierungsphase) soll aus folgenden Inhalten bestehen:
   i. Erarbeitung des finalen Produkts und eine finale Version des Portfolios.
   ii. Kurze Vorstellung der Zielsetzung / Idee / Konzept / Methodik.
   iii. Finale Software der Umsetzung.
   iv. Fragestellung: Welches Ergebnis wurde erreicht? Entspricht das Ergebnis dem Ziel der Arbeit?
   v. Reflexion der Leistung. Welche Schlüsse können gezogen werden.

   Abgabe:

   1. Kurze Erläuterung zur Umsetzung in Form eines Textes mit einem 2-seitigen Abstract und dem finalen Quellcode im ipynb-Format
   2. Phase I.
   3. Phase II.

Verwenden Sie für alle Abstracts wissenschaftliche Fachsprache und stellen Sie sicher, dass sie frei von Rechtschreib-, Zeichensetzungs- und Grammatikfehlern sind. Beachten Sie, dass der finale Abstract ein **Quellenverzeichnis** enthalten muss (Insbesondere für die verwendeten ML-Methoden). Es wird empfohlen, dass Sie sich an den "Richtlinien für die Gestaltung wissenschaftlicher Arbeiten an der IU" orientieren und somit den APA 7 German Zitationsstil konsequent durchsetzen. Quellen sind insbesondere anzugeben, wenn ML-Methoden oder Entwurfsmuster erklärt, begründet oder fachlich eingeordnet werden. Reine Ergebnisdarstellungen und die Projektbeschreibung benötigen in der Regel keine Zitate.
Transparenz und Dokumentation: Der eigene Arbeitsprozess, einschließlich des Einsatzes von KI-Tools, sollte klar dokumentiert werden (Stichwort KI-Disclaimer).
Die Bewertung konzentriert sich stärker auf das kritische Denken und die Methodenkompetenz und Stringenz. Grenzen und potenzielle Fehler von KI-generierten Inhalten müssen erkannt und korrigiert werden.

Verwenden Sie bei der Programmierung mit Python konsequent den PEP 8-Standard, welcher in der Vorlesung diskutiert wird.

In diesem Kurs zählt die geeignete Verwendung und Wiederverwendbarkeit der Softwarelösung (90%). 10% zählt die korrekte Verwendung der ML-Methoden, die nur den Anwendungsbereich der Softwarelösung ausmachen sollen.
Diesbezüglich könnte man sich am Keras-Framework (Chollet, F. (2017). Deep learning with Python. Manning Publications. Chollet, F. (2021). Deep learning with Python(2nd ed.). Manning Publications) von Francois Chollet orientieren, der insbesondere das **Strategy-Pattern** von den GoF sehr stark einsetzt (Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable*

object-oriented software. Addison-Wesley.) Ebenso soll dieses Pattern bei Ihrer Softwarelösung stark zur Geltung kommen.

Das Keras-Framework bietet sich an, da es eines der populärsten Frameworks ist, um mit Deep-Learning mit dem Baukastenprinzip („LEGO-Bricks“) zu arbeiten. Daher sind weitere Software-Patterns aus dem GoF-Buch sicherlich **Composite-Pattern**, um diesen letztgenannten Effekt zu erzielen. Weiterhin **Template**-**Method-Pattern**, um Subklassen zu definieren, um das Verhalten eines Algorithmus zu verändern, **Observer-Pattern**, um zusätzliche Funktionalität anzubieten, ohne den Kern zu verändern (Verstärkender LEGO-Effekt: Zusatzelemente können dynamisch angeklickt und abgezogen werden, ohne den Bau an sich zu destabilisieren. Zusätzlich spielt das **Facade-Pattern** eine Rolle, um Komplexität zu verstecken und einen einzelnen Einstiegspunkt für komplexe Abläufe zu exponieren, letztlich das **Adapter-Pattern**, damit eine einheitliche API über unterschiedliche Implementierungen verwendet werden kann und die Modelle portabel sind.)

Wichtig ist, dass hier Francois Chollet typisch gedacht wird. Das Stichwort ist API-first. Dies bedeutet, dass die GoF-Patterns nicht akademisch, sondern konsequent pragmatisch eingesetzt werden.

Das Portfolio ist so angelegt, dass die konzeptionelle und software-technische Umsetzung (Phase I & II) unabhängig von tiefgehendem ML-Wissen erfolgen kann. Die fachliche Einordnung und Reflexion der eingesetzten ML-Methoden erfolgt erst später in Phase III dann parallel zur Vorlesung, wenn gezielt über die ML-Thematik diskutiert wird.

Hinweis: Auch dann, wenn dies in der Aufgabe nicht explizit gefordert wird und daher optional ist: Eine **Interpretation der Ergebnisse** wird für ein echtes Verständnis der Materie empfohlen.

AUFGABENBESCHREIBUNG FÜR DAS PORTFOLIO

Folgende Machine-Learning-Aufgabe soll in Hinblick auf Verwendbarkeit und Wiederverwendbarkeit, wie in der Vorbemerkung dargestellt, an den GoF-Patterns im Keras-Framework nach Francois Chollet ausgeführt werden, sodass hier auch eine Art „LEGO-Bricks“ Anwendung existiert.

Gegeben sei folgender Python Code:

```
# Codeteil 0

# a)
import numpy as np

# b)
import pandas as pd

# c)
import seaborn as sns

# d)
import matplotlib.pyplot as plt

# e)
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

# Codeteil 1 a)
np.random.seed(123)

# Codeteil 1 b)
iris = sns.load_dataset('iris')

# Codeteil 1 c)
indices = np.random.permutation(len(iris))

# Codeteil 2 a)
test = iris.iloc[indices[:15]]

# Codeteil 2 b)
train = iris.iloc[indices[15:]]

# Codeteil 3 a)
features = ['sepal_length', 'sepal_width', 'petal_length',
'petal_width']
target = 'species'
```

```
# Codeteil 3b)
clf = DecisionTreeClassifier(random_state=123)
clf.fit(train[features], train[target])

# Codeteil 3c)
y_pred = clf.predict(test[features])

# Codeteil 4a)
error_rate = np.mean(y_pred != test[target])
print("Fehlklassifikationsrate:", error_rate)

# Codeteil 5a)


cm = confusion_matrix(test[target], y_pred)


labels = sorted(set(test[target]))


# In DataFrame umwandeln
cm_df = pd.DataFrame(
    cm,
    index=labels,       # Reference (wahre Klassen)
    columns=labels      # Prediction (vorhergesagte Klassen)
)


# Codeteil 5b)
print(cm_df)


plt.figure(figsize=(8, 6))


sns.heatmap(
    cm_df,
    annot=True,                 # Zahlen anzeigen
    fmt="d",
    cmap="Blues",               # ähnlich zu white → skyblue
    cbar=True,
```

```
    linewidths=0.5,
    linecolor="white"
)

plt.title("Konfusionsmatrix: ")
plt.xlabel("Vorhergesagte Klasse")
plt.ylabel("Wahre Klasse")

plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```

Zusätzlich zu diesem Python Code sollen folgende weitere Algorithmen zur Anwendung kommen (nebst Decision Tree Classifier).

a) **Random Forest Klassifikator als Ensemblemethode**
b) **Fisher'sche lineare Diskriminanzanalyse (LDA)**
c) **Verwende eine Support Vector Machine (SVM) RBF Kernel**
d) **Verwende eine SVM mit linearem Kernel**
e) **Verwende eine SVM mit Polynomkern (3. Grad)**
f) **Verwende eine lineare SVM mit linearem Kernel**
g) **XGBoost**

PHASE I – KONZEPTIONSPHASE

- Konzipieren Sie eine Mini-Bibliothek („Mini-Keras"), die den oben gegebenen ML-Workflow abstrahiert. Die in Phase I verwendeten OOP-Konzepte sind nur konzeptionell zu verstehen. Eine konkrete Implementierung in Python ist in dieser Phase noch nicht erforderlich.

- Zielsetzung: Verwendbarer und wiederverwendbarer ML-Workflow statt Skript

- Konzept: Welche Bausteine habe ich? Dataset, Model, Trainer, Metric…
-
- <u>Generelle Forderungen</u>

- Begründen Sie die Auswahl der in Ihrem „Mini-Keras-Projekt“ verwendeten Bibliotheken und deren jeweilige Rolle im Gesamtsystem.
- Warum reicht ein rein skriptbasierter ML-Workflow (wie im gegebenen Code) nicht aus bezüglich Wiederverwendbarkeit und welches Problem wird durch eine Keras-ähnliche Abstraktion ermöglicht?
- Begründen Sie die Auswahl der verwendeten Bibliotheken und ordnen Sie diese konzeptionell der geplanten „Mini-Keras-Architektur“ zu.
- Entwerfen Sie eine modulare Architektur Ihrer „Mini-Keras-Bibliothek“ und visualisieren Sie diese mit einem der Programme, die in den Vorbemerkungen angegeben sind.
- Es ist zu benennen, welche Entwurfsmuster im Projekt eingesetzt werden sollten und wo diese im System vorgesehen sind.

- Es soll ein einheitliches Strategy-Interface für Classifier vorgeben werden (zum Beispiel `ClassifierStrategy` (oder `IClassifier`).
- Der Trainings-/Evaluationscode darf nicht wissen, ob es SVM, LDA oder RF ist.
- Er spricht nur das Classifier-Interface an

- <u>Strategy-Forderung</u>

- Das Strategy-Klassendiagramm soll folgendes zeigen: Trainer hängt von ClassifierStrategy ab mit den konkreten Strategien: `DecisionTreeStrategy (1)`, `RandomForestStrategy (2)`, `LDAStrategy (3)`, `SVCRbfStrategy (4)`, `SVCLinearStrategy (5)`, `SVCPoly3Strategy (6)`, `LinearSVCStrategy (7)`, `XGBoostStrategy (8)`.

- <u>Factory-Forderung</u>

- Das Factory-Klassendiagramm soll zeigen, wie die Strategie ausgewählt wird. Bspw. durch
- `make_classifier(„svm_rbf“, **kwargs)`
- oder `ClassifierFactory.create(config)`

- Der Austausch des Algorithmus muss ohne Änderungen am Trainings- und Evaluationscode möglich sein. Fallunterscheidungen im Trainingsprozess sind nicht zulässig. Die Auswahl erfolgt ausschließlich über **Factory** und das einheitliche Interface.

- <u>Mapping-Tabelle „Algorithmus Strategy“</u>

- Sinnvoll ist hierbei eine kleine Tabelle, die folgende Elemente enthalten soll:
- Algorithmus
- Sklearn/xgboost Klasse
- Geplante Strategy Klasse
- Wichtige Parameter (Kernel/degree/n_estimators)

- <u>Python Sprachkonzepte</u>

- Welche Python-Sprachkonzepte sollen aus der Vorlesung gezielt in der „Mini-Keras-Bibliothek“ eingesetzt werden (Beispiele: List Comprehensions, Generator Expressions / Iteratoren, Funktionen als First-Class Objects, Lambdas, Default-Parameter / *args, **kwargs, einfache Klassen und Methoden)?
- Hinweis: Der Nachweis der Python-Sprachkonzepte kann in Phase II exemplarisch erfolgen.

- <u>API-Design mit Python-Idiomen</u>

- Die geplante API (z.B. `fit`, `predict`, `evaluate`) soll bewusst Python-idiomatisch entworfen werden. Mit den Vorlesungsbezügen: Funktionen & Methoden, klare Parameterübergabe, sinnvolle Default-Werte, saubere Rückgabewerte

PHASE II – ERARBEITUNGS- & REFLEXIONSPHASE

- Zielsetzung: Eine konzeptionell vollständige „Mini-Keras-Bibliothek“ (Schnittstellen /Design), die den angegebenen ML-Workflow kapselt soll abgebildet werden, die die folgende Mindestfunktionalität anbietet:

1. Laden eines Datensatzes
2. Reproduzierbare Aufteilung (Seed!)
3. Training
4. Vorhersage
5. Evaluation

- <u>Verpflichtende Software-Patterns</u>

- Genanntes **Strategy-Pattern** und **Factory-Method-Pattern** zur Implementierung eines austauschbaren Algorithmus müssen in dieser Phase konzeptionell und exemplarisch nachgewiesen werden.

- Die Anwendung von weiteren der oben genannten Entwurfsmuster ist nicht verpflichtend, wird jedoch bei sinnvoller Verwendung bei der Benotung positiv berücksichtigt.

- <u>Verpflichtende Python-Sprachkonzepte</u>

- In Phase II erfolgt der Nachweis exemplarisch durch Code-Snippets, Signaturen oder Pseudocode, die in Phase III dann richtig implementiert werden.
- Mindestens eine Datenverarbeitung oder Auswertung soll mithilfe einer **List Comprehension** oder **Generator Expression** umgesetzt werden.
- Beispiele: Fehlklassifikationsrate, Berechnung der Metrik, Iteration über Vorhersagen

- Mindestens eine Komponente (z.B. Metrik oder Auswertungslogik) muss als Funktion übergeben und gespeichert werden.
- Beispiele: Funktionen als Objekte, Strategy Pattern, Lose Kopplung

- Lambda-Funktionen dürfen dort eingesetzt werden, wo eine kurze, einmalige Funktion sinnvoll ist (z.B. einfache Metriken)

- Die API soll Default-Parameter nutzen
- Reflexion: Welche Python-Sprachkonzepte sind besonders zur Lesbarkeit und Wiederverwendbarkeit zu verwenden.

PHASE III – FINALISIERUNGSPHASE

- Forderung (Phase-3-Artefakt)
- Im 2-seitigen Abstract müssen enthalten sein:

1. Der Vergleich der Strategien (alle 8 in Mapping / Design) ist zu berücksichtigen und zu implementieren. Ein experimenteller Vergleich ist nicht notwendig. Ein minimaler End-to-End-Durchlauf (Train ➔ Predict ➔ Evaluate inkl. Fehlerrate und Confusion Matrix) sollte aber schon demonstriert werden. Dabei müssen alle Strategien instanziierbar sein.
2. Reflexion über Strategy-/Factory-Method-Pattern und andere
3. Begründung, warum ein einheitliches Interface die Wiederverwendbarkeit erhöht